

NLP Assignment 2 Report

NLP Assignment 2 Report

****Name:** Jateen**

****Roll Number:** B22CS026**

****Problem 1: Learning Word Embeddings from IIT Jodhpur Data****

****Problem 2: Character-Level Name Generation using RNN Variants****

PROBLEM 1: Word Embeddings from IIT Jodhpur Data

P1: Corpus Statistics

- ****Size of the corpus file (corpus.txt):** 0.168 MB**
- ****Total number of documents:** 36**
- ****Total tokens:** 25,964**
- ****Vocabulary size:** 3,547**

P1: Curation and Preprocessing Steps

1. ****Step-1: Data Collection**:** Scraped 36 pages from the IIT Jodhpur website (Departments, Research, Academics, Announcements, and Faculty profiles) using `BeautifulSoup`.
2. ****Step-2: Boilerplate Removal**:** Removed website navigation, headers, footers, and non-English lines based on character distribution.
3. ****Step-3: Cleaning**:** Removed URLs, email addresses, excessive punctuation, and non-textual artifacts using regular expressions.
4. ****Step-4: Case Normalization**:** Converted all text to lower-case to ensure consistency in the vocabulary.
5. ****Step-5: Tokenization**:** Tokenized the cleaned text into sentences and then into words using the `nltk` library.
6. ****Step-6: Filtering**:** Filtered out short words and low-frequency terms (frequency < 2) to reduce noise in the embeddings.

P1: Word Embedding (300-dimensional vector for "student")

****Word:** student**

****Vector:****

-0.0332, 0.0768, -0.0540, 0.0166, 0.1287, -0.0581, 0.1325, 0.3132, 0.0152, -0.2531, 0.1639, -0.3321, 0.0136, 0.1670, -0.0625, 0.0303, 0.2647, 0.0302, 0.0870, -0.2013, -0.0363, -0.0993, -0.0402, -0.0409, 0.2379, -0.0981, -0.0079, 0.1399, -0.0009, -0.1955, 0.2793, 0.2148, 0.2573, 0.0491, 0.0061, -0.0867, 0.0691, -0.1670, -0.0555, -0.2244, 0.0210, -0.0878, 0.1224, -0.1436, 0.2938, 0.0395, -0.0657, 0.1391, 0.2132, 0.1279, -0.0335, 0.2173, -0.0258, 0.4945, 0.0994, 0.3428, 0.1370, 0.0780, 0.0195, 0.0295, 0.0804

, 0.2847, 0.1784, 0.0062, -0.2059, 0.1322, 0.0449, 0.0781, -0.0034, 0.1986, -0.0058, 0.1291, 0.1675, -0.2651, 0.2191, 0.0783, -0.1050, 0.0214, -0.1892, -0.0813, -0.0829, 0.1267, -0.0512, 0.3153, 0.1579, 0.1999, 0.0993, -0.1966, 0.1001, 0.3156, 0.1392, -0.0833, -0.1254, 0.1173, 0.0844, -0.0081, 0.0483, -0.1338, 0.2399, -0.0644, -0.0824, -0.0188, -0.1603, 0.0743, -0.0908, -0.1876, -0.0464, 0.1061, -0.0887, -0.0982, -0.2921, 0.0763, 0.0154, 0.2368, -0.0027, 0.0362, 0.0480, -0.0192, 0.0926, -0.3540, 0.1749, -0.1050, 0.0442, -0.0509, -0.0589, 0.0479, -0.1219, -0.2245, 0.2742, 0.2679, 0.0255, -0.1560, -0.0653, -0.1290, 0.0717, 0.4318, -0.0419, -0.1544, -0.1650, -0.0917, 0.0400, -0.2401, -0.1098, -0.0277, -0.1026, -0.2030, -0.0642, 0.0018, 0.0783, -0.2667, -0.0960, -0.1297, 0.0692, 0.0572, 0.0013, 0.0187, -0.2271, -0.0971, -0.0112, -0.0152, -0.1142, 0.0045, -0.0459, 0.0722, 0.1139, 0.3181, 0.0746, -0.0624, -0.0267, 0.1068, -0.2320, -0.0301, -0.0596, 0.1046, -0.0659, 0.0053, -0.0784, -0.0829, -0.0141, 0.1260, 0.0115, 0.0642, 0.0477, 0.0450, -0.1304, 0.1254, 0.1332, 0.0799, 0.1418, 0.0242, 0.3049, -0.0933, -0.1628, 0.1725, -0.0006, -0.3005, 0.0397, 0.0649, -0.1059, 0.0509, -0.0735, -0.0505, -0.1170, 0.0720, 0.1789, -0.0574, 0.1407, 0.1869, 0.1582, 0.0563, -0.1839, 0.0663, -0.0328, -0.2670, -0.0342, -0.0568, 0.0133, -0.4136, -0.1195, -0.1311, -0.0663, 0.0298, -0.1353, -0.0021, -0.1324, -0.1946, -0.0774, 0.1757, -0.0338, -0.0149, 0.1579, -0.0143, 0.0173, 0.1348, -0.1733, 0.0279, -0.1201, -0.0066, 0.0334, -0.1462, 0.1127, -0.1394, -0.1055, 0.1111, 0.0273, -0.0760, 0.2463, -0.1008, 0.2680, 0.3570, 0.0466, -0.1980, 0.1632, -0.0232, -0.2050, -0.0057, 0.1749, -0.0948, -0.5011, -0.1659, -0.1496, -0.0508, 0.1911, -0.1177, -0.4334, 0.1428, 0.0994, 0.0340, -0.0555, 0.0211, 0.0251, 0.0851, 0.2171, 0.1014, -0.0047, 0.0674, 0.1998, 0.0234, 0.0488, -0.1421, 0.0388, -0.0111, -0.0579, 0.3688, 0.0446, -0.2802, -0.1295, -0.0330, -0.0471, 0.1167, 0.2951, 0.0938, 0.1564, -0.0060, 0.1240, 0.1138, 0.0743, 0.0030, 0.0674, -0.2208

(Note: Vector is from the trained Word2Vec Skip-gram model).

P1: Top-10 Words Frequency-wise

the, 1552, of, 1088, and, 773, to, 640, for, 542, in, 482, a, 468, be, 295, student, 238, by, 232

P1: Interesting Analogy

****semester : examination :: thesis : defense****

***Observation:** The model correctly identified that as a semester culminates in an examination, a thesis culminates in a defense.*

PROBLEM 2: Character-Level Name Generation

P2: Quantitative Performance Comparison

Model	Novelty Rate	Diversity	Parameters
:---	:---	:---	:---

	Vanilla RNN		24.6%		80.4%		61,393	
	Bi-LSTM		100.0%		97.6%		442,193	
	Attention + RNN		100.0%		100.0%		271,185	

P2: Evaluation Results

****Which model works better?****

For this specific task of character-level Indian name generation on a relatively small dataset (1000 names), ****Vanilla RNN**** worked significantly better in terms of qualitative realism.

- ****Vanilla RNN**** produces realistic, pronounceable Indian names like **Chaitanya**, **Namit**, and **Vasuki**. Its high memorization (low novelty) is actually a feature here, as it learns the underlying phonotactics and common syllables of Indian names effectively.
- ****Bi-LSTM**** overfitted rapidly (training loss near zero) and generated novel but somewhat random strings (e.g., **Ojw**, **Objs**). While technically expressive, these lack the semantic "look and feel" of the target domain.
- ****Attention + RNN**** was able to capture long-range dependencies but produced stylized/abstract strings that didn't align well with standard name structures on such a small training set.

****Conclusion:**** Vanilla RNN is the most effective for this limited dataset as it captures local character transitions robustly without the over-parameterization issues of Bi-LSTM.

P2: Vanilla RNN Details

- ****Number of Parameters:**** 61,393
- ****Model Size:**** 0.24 MB on disk

Deliverables

- ****GitHub Repository:**** [Link provided in submission]
- ****corpus.txt:**** Preprocessed text for Problem 1
- ****report.pdf:**** This document

(End of Report)